

MACHINE LEARNING

Complete Notes & Reference Guide

From Foundations to Advanced Topics

Mathematics · Theory · Algorithms · Code

For the student who has studied everything — a complete revision companion

1. Foundations of Machine Learning

1.1 What is Machine Learning?

Machine Learning (ML) is a subfield of artificial intelligence that enables systems to learn and improve from experience without being explicitly programmed. The goal is to build algorithms that can access data and use it to learn for themselves.

Three Core Paradigms

Paradigm	Description & Key Algorithms
Supervised Learning	Learn a mapping from inputs $X \rightarrow$ outputs y . Labeled data required. Regression (predict continuous y), Classification (predict discrete y). Examples: Linear Regression, SVM, Random Forest, Neural Networks.
Unsupervised Learning	Find structure/patterns in unlabeled data. No target variable. Examples: K-Means, DBSCAN, PCA, Autoencoders, GMMs.
Reinforcement Learning	An agent learns by taking actions in an environment to maximize cumulative reward. No labeled dataset — feedback is a reward signal. Examples: Q-Learning, PPO, SAC.

The ML Workflow

1. Problem Definition — What are you predicting? Success metric?
2. Data Collection & Storage — Gather raw data from sources.
3. Exploratory Data Analysis (EDA) — Understand distributions, correlations, missing values.
4. Feature Engineering & Preprocessing — Transform raw data into useful features.
5. Model Selection — Choose algorithm(s) appropriate to problem type.
6. Training — Fit model parameters to training data.
7. Evaluation — Measure performance on held-out test data.
8. Hyperparameter Tuning — Optimize model configuration.
9. Deployment — Serve the model in production.
10. Monitoring — Track model performance drift over time.

1.2 Key Mathematical Prerequisites

Linear Algebra Essentials

Vectors & Matrices: An input sample is a vector $x \in \mathbb{R}^n$. A dataset of m samples is a matrix $X \in \mathbb{R}^{m \times n}$ (m rows = samples, n cols = features).

$$X \cdot W = Y \text{ where } X \in \mathbb{R}^{m \times n}, W \in \mathbb{R}^{n \times k}, Y \in \mathbb{R}^{m \times k}$$

Dot product: $x^T y = \sum_i x_i y_i$ — measures similarity between vectors.

Matrix transpose: $(AB)^T = B^T A^T$

Matrix inverse: $AA^{-1} = I$ (only for square, full-rank matrices)

Eigendecomposition: $A = Q\Lambda Q^T$ — fundamental for PCA

$$Av = \lambda v \text{ (} v = \text{eigenvector, } \lambda = \text{eigenvalue)}$$

SVD: $A = U\Sigma V^T$ — generalizes eigendecomposition to non-square matrices. Used in PCA, LSA, recommender systems.

Probability & Statistics

Random Variable: A variable whose value is the outcome of a random phenomenon.

Probability Distributions:

- **Gaussian/Normal:** $X \sim N(\mu, \sigma^2)$ $f(x) = (1/\sqrt{2\pi\sigma^2}) \exp(-(x-\mu)^2/2\sigma^2)$
- **Bernoulli:** $P(X=1) = p$, $P(X=0) = 1-p$
- **Multinomial:** Generalizes Bernoulli to k categories — basis of naive Bayes text classifiers
- **Exponential family:** Covers Gaussian, Bernoulli, Poisson, Gamma — GLMs use these

Bayes' Theorem — the foundation of probabilistic ML:

$$P(\theta | D) = P(D | \theta) \cdot P(\theta) / P(D) \quad [\text{Posterior} \propto \text{Likelihood} \times \text{Prior}]$$

Expectation & Variance:

$$E[X] = \sum x \cdot P(X=x) \quad \text{Var}[X] = E[X^2] - (E[X])^2$$

Covariance & Correlation:

$$\text{Cov}(X,Y) = E[(X-\mu_x)(Y-\mu_y)] \quad \rho = \text{Cov}(X,Y) / (\sigma_x \sigma_y)$$

Calculus for ML

Gradient: $\nabla f(x)$ is a vector of partial derivatives — points in the direction of steepest ascent.

$$\nabla f(x) = [\partial f / \partial x_1, \partial f / \partial x_2, \dots, \partial f / \partial x_n]^T$$

Chain Rule (backpropagation):

$$dL/dW = (dL/da) \cdot (da/dz) \cdot (dz/dW)$$

Jacobian: Matrix of all first-order partial derivatives of a vector-valued function — $J \in \mathbb{R}^{m \times n}$

Hessian: Matrix of second-order partial derivatives — $H \in \mathbb{R}^{n \times n}$. Used in second-order optimization.

Information Theory

Entropy: Measures uncertainty/randomness in a distribution.

$$H(X) = -\sum p(x) \log_2 p(x)$$

KL Divergence: Measures how one distribution P diverges from a reference Q .

$$KL(P \parallel Q) = \sum P(x) \log[P(x)/Q(x)] \quad (\text{always} \geq 0, \text{ not symmetric})$$

Cross-Entropy: Used as loss function for classification:

$$H(P, Q) = -\sum P(x) \log Q(x) = H(P) + KL(P \parallel Q)$$

2. Data Preprocessing & Feature Engineering

2.1 Exploratory Data Analysis (EDA)

EDA is the first step before any modeling. The goal is to understand the data distributions, detect anomalies, and uncover relationships between variables.

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns

df = pd.read_csv('data.csv')

# Shape and types
print(df.shape)           # (rows, cols)
print(df.dtypes)          # dtype per column
print(df.isnull().sum())  # missing count per column

# Statistical summary
print(df.describe())      # count, mean, std, min, quartiles, max

# Correlation heatmap
plt.figure(figsize=(12, 8))
sns.heatmap(df.corr(), annot=True, cmap='coolwarm', center=0)
plt.title('Correlation Matrix')
plt.show()

# Distribution of target variable
df['target'].value_counts().plot(kind='bar')
plt.title('Class Distribution')
plt.show()
```

2.2 Handling Missing Data

Strategy	When to Use
Drop rows/cols (listwise deletion)	When missing rate < 5% and data is MCAR (missing completely at random)
Mean / Median imputation	Numerical data; median for skewed distributions
Mode imputation	Categorical features
KNN imputation	When features are correlated; uses k nearest neighbors
MICE / Iterative imputation	Complex missingness patterns — iteratively imputes each column as a regression target
Indicator variable	Create a binary column '(feature)_missing' to capture missingness pattern itself

```
from sklearn.impute import SimpleImputer, KNNImputer
from sklearn.experimental import enable_iterative_imputer
from sklearn.impute import IterativeImputer

# Median imputation
imp = SimpleImputer(strategy='median')
X_imputed = imp.fit_transform(X)

# KNN imputation
knn_imp = KNNImputer(n_neighbors=5)
```

```
X_knn = knn_imp.fit_transform(X)

# MICE imputation
mice_imp = IterativeImputer(max_iter=10, random_state=42)
X_mice = mice_imp.fit_transform(X)
```

2.3 Feature Scaling & Encoding

Numerical Scaling

Standardization (Z-score): $x' = (x - \mu) / \sigma \rightarrow \text{mean}=0, \text{std}=1$. Required for algorithms using distances or gradients (SVM, PCA, Neural Nets, KNN).

Min-Max Normalization: $x' = (x - \min) / (\max - \min) \rightarrow [0, 1]$ range. Preserves distribution shape. Sensitive to outliers.

Robust Scaler: $x' = (x - Q_2) / (Q_3 - Q_1) \rightarrow$ Uses IQR. Robust to outliers.

Log Transform: $x' = \log(x+1) \rightarrow$ Compresses right-skewed distributions (e.g., income, word counts).

```
from sklearn.preprocessing import StandardScaler, MinMaxScaler, RobustScaler

scaler = StandardScaler() # zero mean, unit variance
X_train_s = scaler.fit_transform(X_train)
X_test_s = scaler.transform(X_test) # NEVER fit on test set!

# Log transform for skewed features
import numpy as np
df['income log'] = np.log1p(df['income']) # log(1+x) handles zeros
```

Categorical Encoding

Encoding	Best For
Label Encoding	Ordinal categories (small < medium < large). Don't use for nominal with tree-based models.
One-Hot Encoding (OHE)	Nominal categories. Drop one column to avoid multicollinearity (get_dummies drop_first=True).
Target (Mean) Encoding	High-cardinality categoricals. Replace category with mean target value. Use with cross-val to prevent leakage.
Frequency Encoding	Replace category with its frequency count. Good for tree-based models.
Embeddings	Very high cardinality (e.g., user IDs). Learn dense vector representations.

```
import pandas as pd
from sklearn.preprocessing import LabelEncoder, OrdinalEncoder

# One-Hot Encoding
df_ohe = pd.get_dummies(df, columns=['color', 'city'], drop_first=True)

# Target Encoding (safe with cross-validation)
# pip install category_encoders
import category_encoders as ce
enc = ce.TargetEncoder(cols=['city'])
df['city_enc'] = enc.fit_transform(df['city'], df['target'])
```

2.4 Feature Engineering

Feature engineering is the process of using domain knowledge to create informative input features that help ML models learn better.

Common Techniques

- **Interaction features:** $f_{1\ 2} = x_1 \times x_2$ — captures multiplicative effects
- **Polynomial features:** x, x^2, x^3 — adds non-linear terms (PolynomialFeatures in sklearn)
- **Binning/Discretization:** Convert continuous → ordinal buckets. Removes noise, adds interpretability.
- **Date/time features:** Extract hour, day-of-week, month, quarter, is_weekend, days_since_event.
- **Text features:** TF-IDF, n-grams, word embeddings (see NLP section).
- **Aggregation features:** For grouped data: mean, std, min, max, count per group.

```
from sklearn.preprocessing import PolynomialFeatures

# Polynomial features up to degree 2
poly = PolynomialFeatures(degree=2, include_bias=False)
X_poly = poly.fit_transform(X)  # x1, x2, x1^2, x1*x2, x2^2

# Date feature extraction
df['hour'] = df['timestamp'].dt.hour
df['day_of_week'] = df['timestamp'].dt.dayofweek
df['is_weekend'] = df['day_of_week'].isin([5, 6]).astype(int)
df['month'] = df['timestamp'].dt.month
```

2.5 Feature Selection

Method	Description
Filter: Variance Threshold	Remove features with variance < threshold (near-constant features). Model-agnostic.
Filter: Correlation	Remove features with $ \text{corr} > 0.95$ with another feature (redundancy). Use corr matrix.
Filter: Mutual Information	$MI(X;Y) = \sum P(x,y) \log[P(x,y)/P(x)P(y)]$ — measures statistical dependence. Works for non-linear.
Wrapper: RFE	Recursive Feature Elimination — train model, remove weakest feature, repeat. Expensive but accurate.
Embedded: L1 Regularization	Lasso drives coefficients to exactly 0 — built-in feature selection for linear models.
Embedded: Feature Importance	Tree-based models output impurity-based or permutation feature importance. Fast and effective.

```
from sklearn.feature_selection import (SelectKBest, f_classif,
    mutual_info_classif, RFE, SelectFromModel)
from sklearn.ensemble import RandomForestClassifier

# ANOVA F-test for classification
sel = SelectKBest(f_classif, k=10)
X_new = sel.fit_transform(X, y)

# Mutual information
sel_mi = SelectKBest(mutual_info_classif, k=10)
X_mi = sel_mi.fit_transform(X, y)

# RFE with logistic regression
from sklearn.linear_model import LogisticRegression
rfe = RFE(LogisticRegression(), n_features_to_select=10)
X_rfe = rfe.fit_transform(X, y)
```

```
# Feature importance from Random Forest
rf = RandomForestClassifier(n_estimators=100)
rf.fit(X_train, y_train)
importances = pd.Series(rf.feature_importances_, index=feature_names)
importances.sort_values(ascending=False).head(15).plot(kind='barh')
```

3. Supervised Learning

3.1 Linear Regression

Goal: Find a linear function $\hat{y} = Xw + b$ that minimizes squared error between predictions and true labels.

Loss Function — Mean Squared Error (MSE)

$$L(w) = (1/m) \sum_i (y_i - \hat{y}_i)^2 = (1/m) \|y - Xw\|^2$$

Closed-Form Solution (Normal Equations)

$$w^* = (X^T X)^{-1} X^T y$$

Complexity: $O(n^3)$ matrix inversion. Infeasible for $n > 10,000$ features. Use gradient descent instead.

Gradient Descent Update Rule

$$w \leftarrow w - \alpha \cdot \nabla L(w) \quad \text{where} \quad \nabla L(w) = -(2/m) X^T (y - Xw)$$

Assumptions of OLS: Linearity, Independence, Homoscedasticity (constant variance), Normality of residuals, No multicollinearity.

Regularization

L2 Ridge: $L = \text{MSE} + \lambda \|w\|^2 \rightarrow$ Shrinks weights toward 0. Closed form: $w^* = (X^T X + \lambda I)^{-1} X^T y$. Never zeroes out features.

L1 Lasso: $L = \text{MSE} + \lambda \|w\|_1 \rightarrow$ Produces sparse solutions (drives some $w = 0$). Built-in feature selection.

Elastic Net: $L = \text{MSE} + \lambda_1 \|w\|_1 + \lambda_2 \|w\|^2 \rightarrow$ Combines L1 and L2. Best of both worlds.

Ridge: $w^* = (X^T X + \lambda I)^{-1} X^T y$ **Lasso:** solved by coordinate descent

```
from sklearn.linear_model import LinearRegression, Ridge, Lasso, ElasticNet
from sklearn.model_selection import cross_val_score
import numpy as np

# Plain OLS
lr = LinearRegression()
lr.fit(X_train, y_train)
print('Coefficients:', lr.coef_)
print('Intercept:', lr.intercept_)
print('R² on test:', lr.score(X_test, y_test))

# Ridge with cross-validated alpha
from sklearn.linear_model import RidgeCV
ridge = RidgeCV(alphas=[0.01, 0.1, 1.0, 10.0, 100.0], cv=5)
ridge.fit(X_train, y_train)
print('Best alpha:', ridge.alpha_)

# Lasso path
from sklearn.linear_model import LassoCV
lasso = LassoCV(cv=5, max_iter=5000)
lasso.fit(X_train, y_train)
print('Nonzero coeffs:', np.sum(lasso.coef_ != 0))
```

3.2 Logistic Regression

Goal: Predict probability $P(y=1|x)$ using the sigmoid function. Despite the name, it's a classification algorithm.

$$\sigma(z) = 1 / (1 + e^{-z}) \quad \text{where } z = w^T x + b$$

Loss Function — Binary Cross-Entropy (Log Loss)

$$L(w) = -(1/m) \sum_i [y_i \log(\hat{y}_i) + (1-y_i) \log(1-\hat{y}_i)]$$

For multi-class, use Softmax + Categorical Cross-Entropy:

$$P(y=k|x) = \exp(w_k^T x) / \sum_j \exp(w_j^T x) \quad [\text{Softmax}]$$

```
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import classification_report, roc_auc_score

# Binary classification
clf = LogisticRegression(C=1.0, penalty='l2', solver='lbfgs', max_iter=1000)
clf.fit(X_train, y_train)

# Probabilities
probs = clf.predict_proba(X_test)[:, 1] # prob of class 1
print('ROC-AUC:', roc_auc_score(y_test, probs))

# Full report
preds = clf.predict(X_test)
print(classification_report(y_test, preds))

# Multi-class with softmax
clf_multi = LogisticRegression(multi_class='multinomial', solver='lbfgs')
clf_multi.fit(X_train, y_train)
```

3.3 Decision Trees

Decision Trees recursively partition the feature space using axis-aligned splits. Each internal node is a feature threshold test; each leaf is a prediction.

Splitting Criteria

Gini Impurity (Classification):

$$\text{Gini}(t) = 1 - \sum_k p_k^2 \quad [0 = \text{pure}, 0.5 = \text{max impurity for binary}]$$

Entropy / Information Gain:

$$\text{IG}(t, A) = H(t) - \sum_v (|t_v|/|t|) \cdot H(t_v)$$

MSE Reduction (Regression): Split minimizes weighted sum of child MSEs.

Key Hyperparameters: max_depth (controls overfitting), min_samples_split, min_samples_leaf, max_features. Always prune or limit depth!

Overfitting Warning: Unpruned trees memorize training data (depth = $\log_2(m)$ fits every point). Always constrain via max_depth or min_samples_leaf.

```
from sklearn.tree import DecisionTreeClassifier,
export_text, plot_tree

dt = DecisionTreeClassifier(
    criterion='gini',
    max_depth=5,
    min_samples_leaf=10,
```

```

    random_state=42
)
dt.fit(X_train, y_train)

# Visualize tree (text format)
print(export_text(dt,
feature_names=feature_names))

# Feature importances
import pandas as pd
fi = pd.Series(dt.feature_importances_,
index=feature_names).sort_values()
fi.plot(kind='barh', title='Feature Importances')

```

3.4 Support Vector Machines (SVM)

SVMs find the hyperplane that maximizes the margin (distance) between the two classes. The support vectors are the training points closest to the decision boundary.

Hard-Margin SVM (linearly separable)

Maximize $2/\|w\|$ subject to $y_i(w^T x_i + b) \geq 1 \quad \forall i$

Soft-Margin SVM (with slack variables ξ)

Minimize $\frac{1}{2}\|w\|^2 + C \sum_i \xi_i$ s.t. $y_i(w^T x_i + b) \geq 1 - \xi_i$, $\xi_i \geq 0$

C is the regularization parameter: large C = small margin, low bias; small C = large margin, high bias.

Kernel Trick

Maps data to a higher-dimensional space where it becomes linearly separable, without explicitly computing the mapping.

Kernel	Formula & Use Case
Linear	$K(x,z) = x^T z$ — use for high-dimensional text data
Polynomial	$K(x,z) = (x^T z + c)^d$ — for polynomial relationships
RBF (Gaussian)	$K(x,z) = \exp(-\gamma \ x - z\ ^2)$ — most common; universal approximator
Sigmoid	$K(x,z) = \tanh(\alpha x^T z + c)$ — neural-net-like

```

from sklearn.svm import SVC, SVR
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import StandardScaler

# SVM pipeline (ALWAYS scale before SVM)
pipe = Pipeline([
    ('scaler', StandardScaler()),
    ('svm', SVC(kernel='rbf', C=1.0, gamma='scale', probability=True))
])
pipe.fit(X_train, y_train)

# Grid search for C and gamma
from sklearn.model_selection import GridSearchCV
param_grid = {'svm__C': [0.1, 1, 10, 100], 'svm__gamma': ['scale', 0.1, 0.01]}
gs = GridSearchCV(pipe, param_grid, cv=5, scoring='f1', n_jobs=-1)
gs.fit(X_train, y_train)
print('Best params:', gs.best_params_)

```

3.5 k-Nearest Neighbors (KNN)

A lazy, instance-based learner. To predict for a new point x , find the k training points nearest to x and return the majority class (classification) or mean (regression).

$$\hat{y} = \text{mode}\{y^{(i)} : i \in K_{nn}(x)\} \quad [\text{classification}] \quad \hat{y} = (1/k) \sum_{i \in K_{nn}(x)} y^{(i)} \quad [\text{regression}]$$

Distance metrics: Euclidean (L2), Manhattan (L1), Minkowski (general L_p), Hamming (categorical), Cosine similarity.

k Selection: Small k = high variance (noisy boundary). Large k = high bias (oversmoothed). Use cross-validation. Typically $k = \sqrt{n}$ is a starting point.

3.6 Naive Bayes

Applies Bayes' theorem with a strong (naive) independence assumption between features given the class label.

$$P(y|x_1, \dots, x_n) \propto P(y) \cdot \prod_i P(x_i|y)$$

Variant	Assumption & Use Case
Gaussian NB	$P(x_i y)$ is Gaussian. For continuous numerical features.
Multinomial NB	$P(x_i y)$ is multinomial. For text (word count vectors / TF-IDF). Very fast.
Bernoulli NB	$P(x_i y)$ is Bernoulli. For binary/boolean features (word presence).
Complement NB	Uses complement classes. Better for imbalanced text datasets.

4. Ensemble Methods

Ensemble methods combine predictions from multiple models to improve accuracy and robustness over any single model.

4.1 Bagging & Random Forest

Bagging (Bootstrap Aggregating): Train B models on B bootstrap samples (sampling with replacement). Average predictions. Reduces variance without increasing bias.

$$\hat{y}_{\text{bag}} = (1/B) \sum_B h_B(x) \quad [\text{regression}] \quad \hat{y}_{\text{bag}} = \text{majority_vote}\{h_B(x)\} \quad [\text{classification}]$$

Random Forest

Extends bagging by also randomizing the feature subset at each split (typically $\sqrt{n}$ for classification, $n/3$ for regression). This decorrelates the trees.

Hyperparameter	Effect
n_estimators	Number of trees. More = better up to diminishing returns. 100-500 typical.
max_depth	Maximum depth of each tree. None = fully grown. Limit for faster training.
max_features	'sqrt', 'log2', or integer. Controls feature subsampling per split.
min_samples_leaf	Minimum samples in a leaf. Higher = more regularization.
oob_score	Out-of-bag samples act as validation — free estimate without hold-out set.

```
from sklearn.ensemble import RandomForestClassifier

rf = RandomForestClassifier(
    n_estimators=500,
    max_features='sqrt',
    max_depth=None,
    min_samples_leaf=2,
    oob_score=True,
    n_jobs=-1,
    random_state=42
)
rf.fit(X_train, y_train)
print('OOB score:', rf.oob_score_)

# Permutation importance (unbiased vs MDI)
from sklearn.inspection import permutation_importance
perm = permutation_importance(rf, X_test, y_test, n_repeats=30)
fi = pd.Series(perm.importances_mean, index=feature_names).sort_values()
```

4.2 Boosting

Boosting: Sequentially train models, each correcting the errors of the previous. Reduces bias. Prone to overfitting (use early stopping).

AdaBoost

Re-weights misclassified samples to force next weak learner to focus on hard examples.

$$H_t(x) = \sum_B \alpha_B h_B(x) \quad \text{where} \quad \alpha_B = \frac{1}{2} \ln((1-\epsilon_B)/\epsilon_B) \quad \text{and} \quad \epsilon_B = \text{weighted error rate}$$

Gradient Boosting (GBM)

Frames boosting as gradient descent in function space. Each tree fits the negative gradient of the loss (residuals for MSE).

$$F_m(x) = F_{\{m-1\}}(x) + \eta \cdot h_m(x) \quad \text{where } h_m \text{ fits } -\partial L / \partial F_{\{m-1\}}(x_i) \text{ for all } i$$

η (learning rate): Shrinkage parameter. Lower $\eta \rightarrow$ need more trees but better generalization. Typically 0.01–0.1.

XGBoost — eXtreme Gradient Boosting

State-of-the-art gradient boosting with second-order Taylor expansion, column subsampling, and regularization.

$$Obj = \sum_i L(y_i, \hat{y}_i) + \sum_k \Omega(f_k) \quad \text{where } \Omega(f) = \gamma T + \frac{1}{2} \lambda \|w\|^2$$

```
import xgboost as xgb
from sklearn.model_selection import cross_val_score

xgb_model = xgb.XGBClassifier(
    n_estimators=500,
    learning_rate=0.05,
    max_depth=6,
    subsample=0.8,           # row subsampling
    colsample_bytree=0.8,    # column subsampling
    reg_lambda=1.0,          # L2 regularization
    reg_alpha=0.0,           # L1 regularization
    use_label_encoder=False,
    eval_metric='logloss',
    random_state=42
)

# Early stopping
xgb_model.fit(
    X_train, y_train,
    eval_set=[(X_val, y_val)],
    early_stopping_rounds=50,
    verbose=False
)

print('Best iteration:', xgb_model.best_iteration)
```

LightGBM & CatBoost

Algorithm	Key Innovation & Notes
LightGBM	Leaf-wise (best-first) tree growth vs level-wise in GBM. Histogram-based splits — much faster. GOSS (Gradient-based One-Side Sampling). Great for large datasets (>100K rows).
CatBoost	Ordered boosting prevents target leakage in categorical encoding. Native handling of categorical features (no manual encoding). Symmetric trees. Often best out-of-the-box.

```
import lightgbm as lgb

lgb_model = lgb.LGBMClassifier(
    n_estimators=1000,
    learning_rate=0.05,
    num_leaves=31,           # max 2^max_depth - 1
    min_child_samples=20,
    subsample=0.8,
    colsample_bytree=0.8,
    reg_lambda=0.1,
    n_jobs=-1,
    random_state=42
)

lgb_model.fit(
    X_train, y_train,
```

```
eval_set=[(X_val, y_val)],
callbacks=[lgb.early_stopping(50), lgb.log_evaluation(100)]
)
```

4.3 Stacking & Blending

Stacking: Train multiple 'base learners' (level-0), then train a 'meta-learner' (level-1) on the base learners' out-of-fold predictions.

Blending: Simpler version — base learners trained on 80%, meta-learner trained on holdout 20% predictions.

```
from sklearn.ensemble import StackingClassifier
from sklearn.linear_model import LogisticRegression
from sklearn.ensemble import RandomForestClassifier, GradientBoostingClassifier
from sklearn.svm import SVC

base_learners = [
    ('rf', RandomForestClassifier(n_estimators=100, random_state=42)),
    ('gb', GradientBoostingClassifier(n_estimators=100, random_state=42)),
    ('svc', SVC(probability=True, kernel='rbf')),
]

stack = StackingClassifier(
    estimators=base_learners,
    final_estimator=LogisticRegression(),
    cv=5,
    passthrough=False # True = also feed original features to meta-learner
)
stack.fit(X_train, y_train)
```

5. Model Evaluation & Validation

5.1 The Bias-Variance Tradeoff

The generalization error of a model decomposes into three terms:

$E[(y - \hat{y})^2] = \text{Bias}^2 + \text{Variance} + \text{Irreducible Noise}$	
Term	Description & Diagnosis
Bias ²	Error from wrong assumptions. High bias = underfitting. Train error $\approx$ Test error, both high. Fix: more complex model, more features.
Variance	Error from sensitivity to training data fluctuations. High variance = overfitting. Train error $\ll$ Test error. Fix: regularization, more data, simpler model.
Irreducible Noise	Inherent randomness in data. Cannot be reduced by any model. Represents the Bayes error rate.

5.2 Cross-Validation Strategies

Strategy	Description & Use Case
k-Fold CV	Split data into k equal folds. Train on k-1, validate on 1. Rotate k times. k=5 or 10 standard. Good general purpose.
Stratified k-Fold	Preserves class ratio in each fold. MUST use for imbalanced classification.
Leave-One-Out (LOO)	k = n. Very low variance, very high computation. Only for tiny datasets.
Time Series CV	Walk-forward validation. Train on past, test on future. NEVER shuffle time series data!
Grouped k-Fold	Ensures samples from same group (patient, user) are in same fold. Prevents data leakage.

```
from sklearn.model_selection import (
    KFold, StratifiedKFold, TimeSeriesSplit, cross_validate)

# Stratified k-fold
skf = StratifiedKFold(n_splits=5, shuffle=True, random_state=42)

results = cross_validate(
    model, X, y,
    cv=skf,
    scoring=['accuracy', 'f1_weighted', 'roc_auc'],
    return_train_score=True
)
print('Test F1:', results['test_f1_weighted'].mean(), '±',
      results['test_f1_weighted'].std())

# Time series split (always train on past, validate on future)
tscv = TimeSeriesSplit(n_splits=5, gap=0)
for train_idx, val_idx in tscv.split(X):
    X_tr, X_val = X[train_idx], X[val_idx]
```

5.3 Classification Metrics

Confusion Matrix

Metric	Formula & Interpretation
Accuracy	$TP+TN / \text{Total}$ — misleading for imbalanced classes
Precision	$TP / (TP+FP)$ — of predicted positives, how many are truly positive? Minimize FP.
Recall (Sensitivity)	$TP / (TP+FN)$ — of actual positives, how many did we catch? Minimize FN.
F1 Score	$2 \cdot \text{Precision} \cdot \text{Recall} / (P+R)$ — harmonic mean; balanced metric for imbalanced classes
ROC-AUC	Area under ROC curve (TPR vs FPR). AUC=0.5 random, AUC=1.0 perfect. Threshold-invariant.
PR-AUC	Area under Precision-Recall curve. Better than ROC-AUC for highly imbalanced datasets.
MCC	$(TP \cdot TN - FP \cdot FN) / \sqrt{(TP+FP)(TP+FN)(TN+FP)(TN+FN)}$ — robust single metric, works for multiclass

```
from sklearn.metrics import (
    confusion_matrix, classification_report,
    roc_auc_score, average_precision_score,
    matthews_corrcoef, f1_score)
import matplotlib.pyplot as plt
import seaborn as sns

y_pred = model.predict(X_test)
y_prob = model.predict_proba(X_test)[:, 1]

cm = confusion_matrix(y_test, y_pred)
sns.heatmap(cm, annot=True, fmt='d', cmap='Blues')
plt.xlabel('Predicted'); plt.ylabel('Actual'); plt.show()

print(classification_report(y_test, y_pred, digits=4))
print('ROC-AUC:', roc_auc_score(y_test, y_prob))
print('PR-AUC: ', average_precision_score(y_test, y_prob))
print('MCC:    ', matthews_corrcoef(y_test, y_pred))

# ROC Curve
from sklearn.metrics import RocCurveDisplay
RocCurveDisplay.from_predictions(y_test, y_prob)
plt.title('ROC Curve'); plt.show()
```

5.4 Regression Metrics

Metric	Formula & Notes
MAE	$\sum y_i - \hat{y}_i / m$ — robust to outliers, in target units
MSE / RMSE	$\sum (y_i - \hat{y}_i)^2 / m$ — penalizes outliers heavily. RMSE in same units as target.
R ² (R-squared)	$1 - SS_{\text{res}} / SS_{\text{tot}}$ — proportion of variance explained. 1.0 = perfect, 0 = baseline mean model.
MAPE	$\sum y_i - \hat{y}_i / y_i / m$ — percentage error. Undefined for $y_i=0$. Use sMAPE as alternative.
Huber Loss	MSE for $ err < \delta$, MAE otherwise. Combines stability of MSE and outlier-robustness of MAE.

5.5 Hyperparameter Tuning

Method	Description & Pros/Cons
Grid Search	Exhaustive search over parameter grid. Guaranteed to find best in grid. Scales poorly (combinatorial).
Random Search	Sample parameters randomly. Empirically finds good params in 60x fewer evaluations than grid search.
Bayesian Optimization	Build probabilistic model of objective. Balance exploration vs exploitation. Optuna, Hyperopt, BoTorch.
Successive Halving	Train on small budget, discard worst, repeat. Very efficient. Sklearn HalvingGridSearchCV.
AutoML (TPOT, H2O)	Automate feature engineering + model selection + hyperparameter tuning simultaneously.

```
import optuna
optuna.logging.set_verbosity(optuna.logging.WARNING)

def objective(trial):
    params = {
        'n_estimators' : trial.suggest_int('n_estimators', 100, 1000),
        'max_depth' : trial.suggest_int('max_depth', 3, 10),
        'learning_rate' : trial.suggest_float('learning_rate', 0.01, 0.3,
log=True),
        'subsample' : trial.suggest_float('subsample', 0.5, 1.0),
        'colsample_bytree': trial.suggest_float('colsample_bytree', 0.5, 1.0),
    }
    model = xgb.XGBClassifier(**params, random_state=42, eval_metric='logloss')
    cv_scores = cross_val_score(model, X_train, y_train, cv=5,
                                scoring='roc_auc', n_jobs=-1)

    return cv_scores.mean()

study = optuna.create_study(direction='maximize')
study.optimize(objective, n_trials=100, n_jobs=-1)
print('Best params:', study.best_params)
print('Best ROC-AUC:', study.best_value)
```

6. Unsupervised Learning

6.1 K-Means Clustering

Partitions n observations into k clusters by minimizing within-cluster sum of squared distances to centroids.

$$J = \sum_k \sum_{x \in C_k} \|x - \mu_k\|^2 \quad [\text{objective to minimize}]$$

Algorithm (Lloyd's)

11. Initialize k centroids randomly (or with k-means++ for better initialization).
12. Assign each point to nearest centroid: $c_i = \operatorname{argmin}_k \|x_i - \mu_k\|^2$
13. Update centroids: $\mu_k = (1/|C_k|) \sum_{x \in C_k} x$
14. Repeat 2-3 until convergence (no reassignments).

Choosing k : Elbow method (plot inertia vs k), Silhouette score (higher = better separation, range $[-1, 1]$).

Silhouette(i) = (b(i) - a(i)) / max(a(i), b(i)) where **a**=mean intra-cluster dist, **b**=mean nearest-cluster dist

```
from sklearn.cluster import KMeans
from sklearn.metrics import silhouette_score
import matplotlib.pyplot as plt

# Elbow method
inertias = []
K_range = range(2, 12)
for k in K_range:
    km = KMeans(n_clusters=k, init='k-means++', n_init=10, random_state=42)
    km.fit(X_scaled)
    inertias.append(km.inertia_)

plt.plot(K_range, inertias, 'bo-')
plt.xlabel('k'); plt.ylabel('Inertia'); plt.title('Elbow Method')
plt.show()

# Silhouette analysis
sil_scores = [silhouette_score(X_scaled, KMeans(n_clusters=k,
random_state=42).fit_predict(X_scaled))
               for k in K_range]
best_k = K_range[sil_scores.index(max(sil_scores))]
print('Best k by silhouette:', best_k)
```

6.2 DBSCAN

Density-Based Spatial Clustering of Applications with Noise. Finds clusters of arbitrary shape. Identifies outliers as noise points.

Parameters: ϵ (epsilon) = neighborhood radius, min_samples = minimum points to form a core point.

Point Type	Definition
Core point	Has $\geq$ min_samples points within ϵ radius (including itself)
Border point	Within ϵ of a core point but $<$ min_samples neighbors itself
Noise point	Neither core nor border. Labeled as -1. These are outliers.

```
from sklearn.cluster import DBSCAN
from sklearn.preprocessing import StandardScaler
```

```

X_scaled = StandardScaler().fit_transform(X)
db = DBSCAN(eps=0.5, min_samples=5, metric='euclidean', n_jobs=-1)
labels = db.fit_predict(X_scaled)

n_clusters = len(set(labels)) - (1 if -1 in labels else 0)
n_noise = list(labels).count(-1)
print(f'Clusters: {n_clusters}, Noise points: {n_noise}')

# Visualize (2D after PCA)
from sklearn.decomposition import PCA
X_2d = PCA(n_components=2).fit_transform(X_scaled)
plt.scatter(X_2d[:,0], X_2d[:,1], c=labels, cmap='tab10', s=10)
plt.title('DBSCAN Clusters'); plt.show()

```

6.3 Principal Component Analysis (PCA)

PCA finds the orthogonal directions of maximum variance in data and projects to a lower-dimensional subspace. Used for dimensionality reduction, visualization, and noise removal.

Mathematical Derivation

15. Standardize data: $X_scaled = (X - \mu) / \sigma$
16. Compute covariance matrix: $C = (1/m) X^T X \in \mathbb{R}^{n \times n}$
17. Eigendecomposition: $C = Q \Lambda Q^T$ (eigenvectors in columns of Q , eigenvalues in Λ)
18. Sort eigenvectors by decreasing eigenvalue.
19. Project: $Z = XQ_k$ where Q_k = first k eigenvectors (principal components)

Explained Variance Ratio = $\lambda_k / \sum_i \lambda_i$ [fraction of variance explained by PC k]

```

from sklearn.decomposition import PCA
import numpy as np

pca = PCA(n_components=0.95) # keep 95% variance
X_pca = pca.fit_transform(X_scaled)
print(f'Components: {pca.n_components_}')
print(f'Explained variance ratio: {pca.explained_variance_ratio_.cumsum() [-1]:.3f}')
```

```

# Scree plot
plt.plot(np.cumsum(pca.explained_variance_ratio_))
plt.xlabel('Number of Components')
plt.ylabel('Cumulative Explained Variance')
plt.axhline(y=0.95, color='r', linestyle='--', label='95% threshold')
plt.title('PCA Scree Plot'); plt.legend(); plt.show()
```

```

# 2D visualization
pca2 = PCA(n_components=2)
X_2d = pca2.fit_transform(X_scaled)
plt.scatter(X_2d[:,0], X_2d[:,1], c=y, cmap='tab10', alpha=0.7)
plt.xlabel('PC1'); plt.ylabel('PC2'); plt.title('PCA 2D Projection')

```

6.4 t-SNE & UMAP

Method	Description & Notes
--------	---------------------

t-SNE	t-Distributed Stochastic Neighbor Embedding. Models pairwise similarities as probabilities, minimizes KL divergence. Excellent for visualization but non-deterministic, slow for large datasets. Use perplexity 5-50.
UMAP	Uniform Manifold Approximation and Projection. Preserves global structure better than t-SNE. Much faster. Supports supervised/semi-supervised learning. Preferred for most visualization tasks.

```

from sklearn.manifold import TSNE
import umap # pip install umap-learn

# t-SNE
tsne = TSNE(n_components=2, perplexity=30, learning_rate='auto',
            init='pca', random_state=42)
X_tsne = tsne.fit_transform(X_scaled)

# UMAP
reducer = umap.UMAP(n_components=2, n_neighbors=15, min_dist=0.1, random_state=42)
X_umap = reducer.fit_transform(X_scaled)

fig, axes = plt.subplots(1, 2, figsize=(14, 5))
for ax, X_2d, title in zip(axes, [X_tsne, X_umap], ['t-SNE', 'UMAP']):
    ax.scatter(X_2d[:,0], X_2d[:,1], c=y, cmap='tab10', s=5, alpha=0.7)
    ax.set_title(title)
plt.tight_layout(); plt.show()

```

7. Neural Networks & Deep Learning

7.1 Feedforward Neural Networks (MLP)

A Multi-Layer Perceptron is a series of linear transformations interleaved with non-linear activation functions.

$$\text{Layer } l: \mathbf{a}^{(l)} = \mathbf{f}(\mathbf{W}^{(l)} \mathbf{a}^{(l-1)} + \mathbf{b}^{(l)})$$

Activation Functions

Activation	Formula & Properties
Sigmoid	$\sigma(x) = 1/(1+e^{-x}) \in (0,1)$. Saturates $\rightarrow$ vanishing gradient. Use only for output layer in binary classification.
Tanh	$\tanh(x) = (e^x - e^{-x})/(e^x + e^{-x}) \in (-1,1)$. Zero-centered. Still saturates at extremes.
ReLU	$\max(0, x)$. Sparse activation. No saturation for $x > 0$. Default for hidden layers. Dying ReLU problem.
Leaky ReLU	$\max(\alpha x, x)$, $\alpha \approx 0.01$. Fixes dying ReLU by allowing small negative gradient.
ELU	x if $x > 0$, $\alpha(e^x - 1)$ if $x < 0$. Smooth negative region. Slightly better than ReLU in practice.
GELU	$x \cdot \Phi(x)$ where Φ =CDF of normal. Default in Transformers (BERT, GPT). Smooth and probabilistic.
Softmax	$\exp(x_i) / \sum_j \exp(x_j)$. Converts logits to probability distribution. Output layer for multi-class.

7.2 Backpropagation

Backpropagation efficiently computes gradients using the chain rule of calculus, propagating error backward through the network.

Forward Pass

$$\mathbf{z}^{(l)} = \mathbf{W}^{(l)} \mathbf{a}^{(l-1)} + \mathbf{b}^{(l)} \quad \mathbf{a}^{(l)} = \mathbf{f}(\mathbf{z}^{(l)})$$

Backward Pass (chain rule)

$$\delta^{(L)} = \nabla_{\mathbf{a}} L \odot \mathbf{f}'(\mathbf{z}^{(L)}) \quad [\text{output layer error}]$$

$$\delta^{(l)} = (\mathbf{W}^{(l+1)})^T \delta^{(l+1)} \odot \mathbf{f}'(\mathbf{z}^{(l)}) \quad [\text{hidden layer error}]$$

$$\partial L / \partial \mathbf{W}^{(l)} = \delta^{(l)} (\mathbf{a}^{(l-1)})^T \quad \partial L / \partial \mathbf{b}^{(l)} = \delta^{(l)}$$

7.3 Optimizers

Optimizer	Update Rule & Notes
SGD	$\mathbf{w} \leftarrow \mathbf{w} - \eta \cdot \nabla L$. Noisy updates. Use momentum (SGD+M) in practice. Good final convergence.

SGD + Momentum	$v \leftarrow \beta v - \eta \nabla L; w \leftarrow w + v.$ $\beta \approx 0.9$ damps oscillations. Nesterov variant looks ahead.
Adam	Adaptive Learning Rates. Combines momentum (1st moment) + RMSProp (2nd moment). $m \leftarrow \beta_1 m + (1-\beta_1)g; v \leftarrow \beta_2 v + (1-\beta_2)g^2; w \leftarrow w - \eta \cdot \hat{m} / \sqrt{\hat{v} + \epsilon}.$ Defaults: $\beta_1 = 0.9, \beta_2 = 0.999, \epsilon = 1e-8.$ Standard choice.
AdamW	Adam with decoupled weight decay. Fixes L2 regularization in Adam. Default for Transformers.
RMSProp	$v \leftarrow \beta v + (1-\beta)g^2; w \leftarrow w - \eta \cdot g / \sqrt{v + \epsilon}.$ Good for RNNs.

7.4 Regularization Techniques

Dropout

Randomly sets a fraction p of activations to 0 during training. Prevents co-adaptation of neurons. At test time, scale activations by $(1-p)$ or use inverted dropout.

$$\tilde{a}^{(l)} = (a^{(l)} \odot m) / (1-p) \quad m \sim \text{Bernoulli}(1-p)$$

Batch Normalization

Normalizes activations within a mini-batch. Reduces internal covariate shift, enables higher learning rates, acts as regularizer.

$$\text{BN}(x) = \gamma \cdot (x - \mu_B) / \sqrt{(\sigma^2_B + \epsilon)} + \beta \quad [\gamma, \beta \text{ are learned parameters}]$$

Early Stopping

Monitor validation loss. Stop training when validation loss starts increasing (patience = number of epochs to wait before stopping).

```
import torch
import torch.nn as nn
import torch.optim as optim

class MLP(nn.Module):
    def __init__(self, input_dim, hidden_dims, output_dim, dropout=0.3):
        super().__init__()
        layers = []
        dims = [input_dim] + hidden_dims
        for i in range(len(dims)-1):
            layers += [nn.Linear(dims[i], dims[i+1]),
                      nn.BatchNorm1d(dims[i+1]),
                      nn.ReLU(),
                      nn.Dropout(dropout)]
        layers.append(nn.Linear(dims[-1], output_dim))
        self.net = nn.Sequential(*layers)

    def forward(self, x):
        return self.net(x)

model = MLP(input_dim=128, hidden_dims=[256, 128, 64], output_dim=10)
optimizer = optim.AdamW(model.parameters(), lr=1e-3, weight_decay=1e-4)
criterion = nn.CrossEntropyLoss()

# Training loop with early stopping
best_val_loss = float('inf')
patience, patience_counter = 15, 0
```

```

for epoch in range(300):
    model.train()
    for X_batch, y_batch in train_loader:
        optimizer.zero_grad()
        loss = criterion(model(X_batch), y_batch)
        loss.backward()
        nn.utils.clip_grad_norm_(model.parameters(), 1.0) # gradient clipping
        optimizer.step()

    model.eval()
    with torch.no_grad():
        val_loss = criterion(model(X_val_tensor), y_val_tensor).item()

    if val_loss < best_val_loss:
        best_val_loss = val_loss
        torch.save(model.state_dict(), 'best_model.pt')
        patience_counter = 0
    else:
        patience_counter += 1
        if patience_counter >= patience:
            print(f'Early stop at epoch {epoch}'); break

```

7.5 Weight Initialization

Scheme	Formula & Use Case
Xavier/Glorot	$W \sim \text{Uniform}(-\sqrt{6/(n_i + n_{ou})}, +\sqrt{6/(n_i + n_{ou})})$. For Tanh/Sigmoid activations.
He Initialization	$W \sim N(0, 2/n_i)$. For ReLU activations. Most commonly used.
LeCun	$W \sim N(0, 1/n_i)$. For SELU activation. Preserves mean and variance for self-normalizing nets.

8. Convolutional Neural Networks (CNNs)

8.1 Convolution Operation

A convolution applies a learned filter/kernel to local patches of the input, sharing weights across spatial locations.

$$(I * K)[i,j] = \sum_m \sum_n I[i+m, j+n] \cdot K[m,n]$$

Key Concepts

- **Padding:** 'same' — output same size as input. 'valid' — no padding, output shrinks.
- **Stride:** Step size of the sliding window. Stride=2 halves spatial dimensions.
- **Channels:** For input (H,W,C_in) and F filters of size (k,k,C_in): output has (H',W',F) channels.
- **Receptive field:** The region of input that influences a given output unit. Grows with depth.

$$\text{Output size: } H_{\text{out}} = \lfloor (H + 2P - K) / S \rfloor + 1$$

Pooling

Max Pooling: Takes maximum value in each window. Provides translation invariance. Most common.

Average Pooling: Takes average. Global Average Pooling before classifier replaces flattening.

Adaptive Pooling: Output size is fixed regardless of input size. torch.nn.AdaptiveAvgPool2d((1,1)).

8.2 CNN Architectures

Architecture	Year & Key Innovation
LeNet-5	1998 — First successful CNN. 2 conv + 3 FC. Digit recognition on MNIST.
AlexNet	2012 — Deep CNN with ReLU, Dropout, GPU training. Won ImageNet. Started deep learning revolution.
VGGNet	2014 — Very deep (16-19 layers) with only 3×3 filters. Simple but very effective.
GoogLeNet/Inception	2014 — Inception module: parallel 1×1, 3×3, 5×5 convolutions. Efficient depth.
ResNet	2015 — Residual connections: F(x)+x. Enables training of very deep nets (50, 101, 152 layers). Solved vanishing gradient.
DenseNet	2017 — Dense connections: each layer connects to ALL subsequent layers. Maximum feature reuse.
EfficientNet	2019 — Compound scaling of depth/width/resolution via NAS. State-of-the-art efficiency.
ConvNeXt	2022 — Modernized pure CNN matching ViT performance. Key: depthwise conv, GELU, LayerNorm.

```
import torch.nn as nn
import torchvision.models as models

# Transfer learning with ResNet-50
model = models.resnet50(weights='IMAGENET1K_V2')

# Freeze all layers except last
for param in model.parameters():
    param.requires_grad = False
```

```

# Replace classifier head
num_features = model.fc.in_features
model.fc = nn.Sequential(
    nn.Linear(num_features, 256),
    nn.ReLU(), nn.Dropout(0.4),
    nn.Linear(256, num_classes)
)

# Only train new head initially
optimizer = torch.optim.Adam(model.fc.parameters(), lr=1e-3)

# Fine-tune: unfreeze all after head converges
for param in model.parameters():
    param.requires_grad = True
optimizer = torch.optim.AdamW(model.parameters(), lr=1e-5, weight_decay=1e-4)

```

Residual Block (ResNet)

$$F(x) = H(x) - x \rightarrow H(x) = F(x) + x \quad [\text{skip connection adds identity}]$$

```

class ResidualBlock(nn.Module):
    def __init__(self, channels):
        super().__init__()
        self.conv1 = nn.Conv2d(channels, channels, 3, padding=1, bias=False)
        self.bn1 = nn.BatchNorm2d(channels)
        self.conv2 = nn.Conv2d(channels, channels, 3, padding=1, bias=False)
        self.bn2 = nn.BatchNorm2d(channels)
        self.relu = nn.ReLU(inplace=True)

    def forward(self, x):
        residual = x
        out = self.relu(self.bn1(self.conv1(x)))
        out = self.bn2(self.conv2(out))
        out += residual # skip connection
        return self.relu(out)

```

9. Sequence Models: RNNs, LSTMs & Transformers

9.1 Recurrent Neural Networks (RNN)

RNNs process sequences by maintaining a hidden state that gets updated at each time step.

$$h_t = \tanh(W_h \cdot h_{t-1} + W_x x_t + b) \quad y_t = W_y h_t$$

Vanishing Gradient Problem: In long sequences, gradients for early time steps become exponentially small (or explode). BPTT (Backpropagation Through Time) suffers from this. Solution: LSTM, GRU.

9.2 LSTM (Long Short-Term Memory)

LSTMs use gating mechanisms to control information flow, enabling learning of long-term dependencies.

$$f_t = \sigma(W_f \cdot [h_{t-1}, x_t] + b_f) \quad [\text{Forget gate}]$$

$$i_t = \sigma(W_i \cdot [h_{t-1}, x_t] + b_i) \quad [\text{Input gate}]$$

$$\tilde{g}_t = \tanh(W_g \cdot [h_{t-1}, x_t] + b_g) \quad [\text{Candidate cell state}]$$

$$C_t = f_t \odot C_{t-1} + i_t \odot \tilde{g}_t \quad [\text{Cell state update}]$$

$$o_t = \sigma(W_o \cdot [h_{t-1}, x_t] + b_o) \quad [\text{Output gate}]$$

$$h_t = o_t \odot \tanh(C_t) \quad [\text{Hidden state output}]$$

9.3 The Transformer Architecture

Transformers replaced RNNs in NLP tasks. They rely entirely on attention mechanisms and process sequences in parallel.

Self-Attention (Scaled Dot-Product Attention)

$$\text{Attention}(Q, K, V) = \text{softmax}(QK^T / \sqrt{d_k}) \cdot V$$

Q (queries), K (keys), V (values) are linear projections of the input. The scaling $\sqrt{d_k}$ prevents softmax saturation in high dimensions.

Multi-Head Attention

$$\text{MHA}(Q, K, V) = \text{Concat}(\text{head}_1, \dots, \text{head}_h) \cdot W^O \quad \text{head}_i = \text{Attention}(Q \cdot W_i^Q, K \cdot W_i^K, V \cdot W_i^V)$$

Transformer Block

20. Multi-Head Self-Attention → Add & LayerNorm

21. Feed-Forward Network (2-layer MLP, GELU) → Add & LayerNorm

Positional Encoding: Since Transformers have no recurrence, position info is added via sinusoidal functions:

$$\text{PE}(\text{pos}, 2i) = \sin(\text{pos}/10000^{(2i/d)}) \quad \text{PE}(\text{pos}, 2i+1) = \cos(\text{pos}/10000^{(2i/d)})$$

```
import torch
import torch.nn as nn
import torch.nn.functional as F

class MultiHeadAttention(nn.Module):
    def __init__(self, d_model, num_heads):
        super().__init__()
        self.d_model = d_model
        self.h = num_heads
        self.d_k = d_model // num_heads
        self.Wq = nn.Linear(d_model, d_model)
        self.Wk = nn.Linear(d_model, d_model)
        self.Wv = nn.Linear(d_model, d_model)
        self.Wo = nn.Linear(d_model, d_model)

    def forward(self, Q, K, V, mask=None):
        B, T, _ = Q.size()
        Q = self.Wq(Q).view(B, T, self.h, self.d_k).transpose(1,2)
        K = self.Wk(K).view(B, T, self.h, self.d_k).transpose(1,2)
        V = self.Wv(V).view(B, T, self.h, self.d_k).transpose(1,2)
        scores = (Q @ K.transpose(-2,-1)) / (self.d_k ** 0.5)
        if mask is not None:
            scores = scores.masked_fill(mask == 0, -1e9)
        attn = F.softmax(scores, dim=-1)
        out = (attn @ V).transpose(1,2).contiguous().view(B, T, self.d_model)
        return self.Wo(out)
```

9.4 Transformer Variants

Model	Type & Key Idea
BERT	Encoder-only. Masked Language Model (MLM) + Next Sentence Prediction. Bidirectional context. Fine-tune for NLU tasks.
GPT family	Decoder-only. Causal (left-to-right) language modeling. Scale to huge sizes (GPT-4). Few-shot prompting emergent at scale.
T5	Encoder-decoder. Treats all NLP as text-to-text. Unified framework for translation, summarization, QA.
ViT	Vision Transformer. Splits image into patches → treat as tokens → standard Transformer encoder.
Llama / Mistral	Open decoder-only LLMs. RoPE (rotary) positional encoding, SwiGLU activation, grouped-query attention.

10. Advanced Topics

10.1 Generative Models

Variational Autoencoders (VAE)

VAEs learn a latent representation $z \sim N(\mu, \sigma^2)$ and can generate new data by sampling from the latent space.

$$\text{ELBO} = E[\log P(x|z)] - \text{KL}(Q(z|x) \parallel P(z)) \quad [\text{Maximize this lower bound}]$$

Encoder: $q(z|x) = N(\mu(x), \sigma^2(x))$ — outputs mean and variance of latent distribution.

Reparameterization trick: $z = \mu + \sigma \odot \epsilon, \epsilon \sim N(0, I)$ — makes sampling differentiable.

Decoder: $p(x|z)$ — reconstructs input from latent vector.

Generative Adversarial Networks (GAN)

Two networks compete: Generator G creates fake data; Discriminator D tries to distinguish real from fake.

$$\min_G \max_D E[\log D(x)] + E[\log(1 - D(G(z)))]$$

GAN Variant	Key Innovation
DCGAN	Deep Convolutional GAN. BatchNorm, strided convolutions, no FC layers. Stable baseline.
WGAN	Wasserstein distance instead of JS divergence. More stable training, meaningful loss metric.
StyleGAN2	Style-based generator with noise injection. State-of-the-art face synthesis.
Conditional GAN	Condition G and D on class labels c. Controllable generation.

Diffusion Models

Learn to reverse a gradual noising process. State-of-the-art for image, audio, and video generation.

Forward process: $q(x_{\square} | x_{\square-1}) = N(x_{\square}; \sqrt{(1-\beta_{\square})}x_{\square-1}, \beta_{\square}I)$ — gradually add noise over T steps.

Reverse process: $p_{\theta}(x_{\square-1} | x_{\square}) = N(x_{\square-1}; \mu_{\theta}(x_{\square}, t), \Sigma_{\theta}(x_{\square}, t))$ — learned denoising step.

Training objective: Predict the noise ϵ added at each step: $L = \|\epsilon - \epsilon_{\theta}(x_{\square}, t)\|^2$

10.2 Reinforcement Learning (RL)

An agent interacts with an environment, observing states $s_{\square}$ and taking actions $a_{\square}$ to maximize cumulative reward $R_{\square} = \sum \gamma^k r_{\square+k}$.

Concept	Definition
State (s)	Current situation observed by the agent
Action (a)	Decision taken by the agent
Reward (r)	Scalar feedback signal from environment
Policy (π)	$\pi(a s)$ — probability of taking action a in state s
Value function	$V^{\pi}(s) = E[\sum \gamma^k r_{\square+k} \mid s_{\square}=s]$ — expected return from state s under π
Q-function	$Q^{\pi}(s,a) = E[R_{\square} \mid s_{\square}=s, a_{\square}=a]$ — expected return for state-action pair

Bellman equation	$V^*(s) = \max_a [r(s,a) + \gamma \sum P(s' s,a) V^*(s')]$
------------------	--

Key Algorithms

- **Q-Learning:** $Q(s,a) \leftarrow Q(s,a) + \alpha[r + \gamma \cdot \max_{a'} Q(s',a') - Q(s,a)]$ — model-free, off-policy.
- **DQN:** Deep Q-Network. Q-function as neural net. Experience replay + target network for stability.
- **Policy Gradient (REINFORCE):** $\nabla J(\theta) = E[\nabla \log \pi(a|s) \cdot R]$ — directly optimize policy.
- **PPO:** Proximal Policy Optimization. Clips policy update ratio for stability. Default RL algorithm for LLM fine-tuning (RLHF).
- **SAC:** Soft Actor-Critic. Maximum entropy RL. Entropy term encourages exploration. Best for continuous control.

10.3 Transfer Learning & Fine-Tuning

Use a model pre-trained on large data as a starting point for a smaller downstream task.

Fine-Tuning Strategies

Strategy	Description
Full fine-tuning	Unfreeze and update all parameters. Needs more data and compute. Risk of catastrophic forgetting.
Feature extraction	Freeze pre-trained backbone, only train a new head. Fast, less data needed.
Layer-wise LR decay	Use smaller LR for early layers (general features), larger for later layers (task-specific).
LoRA	Low-Rank Adaptation. Inject small trainable matrices into attention weights. Very parameter-efficient. $\Delta W = BA$, $r \ll d$.
Prompt tuning	Prepend learnable soft tokens to input. Only tune prompt, freeze everything else.

```
# LoRA implementation concept
from peft import LoraConfig, get_peft_model
from transformers import AutoModelForSequenceClassification

base_model = AutoModelForSequenceClassification.from_pretrained('bert-base-uncased')

lora_config = LoraConfig(
    r=16,                                # rank of adaptation matrices
    lora_alpha=32,                        # scaling factor
    target_modules=['query', 'key', 'value'],
    lora_dropout=0.05,
    bias='none',
    task_type='SEQ_CLS'
)

model = get_peft_model(base_model, lora_config)
model.print_trainable_parameters()
# Trainable params: ~0.1% of total - 100x fewer than full fine-tuning!
```

10.4 Explainability (XAI)

Method	Description & Use Case
--------	------------------------

SHAP	SHapley Additive exPlanations. Game-theory based. SHAP values = contribution of each feature to prediction. Works for any model. TreeSHAP is $O(TLD^2)$ for trees.
LIME	Local Interpretable Model-agnostic Explanations. Fit simple model (linear) around a single prediction. Perturb input, see which features matter locally.
Integrated Gradients	Attribute prediction to input features by integrating gradients from baseline to input. For neural networks and images.
Attention Visualization	For Transformers: visualize attention weights to see which tokens influence prediction. Caution: attention $\neq$ explanation.
Partial Dependence Plots	Show marginal effect of one or two features on predicted outcome. For global model understanding.

```
import shap

# SHAP for tree models (fastest)
explainer = shap.TreeExplainer(xgb_model)
shap_values = explainer.shap_values(X_test)

# Summary plot (global feature importance)
shap.summary_plot(shap_values, X_test, feature_names=feature_names)

# Force plot (local explanation for one prediction)
shap.force_plot(explainer.expected_value, shap_values[0], X_test[0],
                feature_names=feature_names)

# For neural networks
explainer_nn = shap.DeepExplainer(model, X_background)
shap_values_nn = explainer_nn.shap_values(X_test[:100])
```

11. MLOps & Production ML

11.1 ML Pipeline Best Practices

Data Leakage — The Silent Killer

Critical Warning: Never fit any preprocessor (scaler, imputer, encoder, feature selector) on test data. Always fit on training data, then transform test. Wrap in sklearn Pipeline to enforce this automatically.

```
from sklearn.pipeline import Pipeline
from sklearn.compose import ColumnTransformer
from sklearn.preprocessing import StandardScaler,
OneHotEncoder
from sklearn.impute import SimpleImputer

numeric_pipeline = Pipeline([
    ('imputer', SimpleImputer(strategy='median')),
    ('scaler', StandardScaler())
])

categorical_pipeline = Pipeline([
    ('imputer',
SimpleImputer(strategy='most_frequent')),
    ('encoder',
OneHotEncoder(handle_unknown='ignore',
sparse_output=False))
])

preprocessor = ColumnTransformer([
    ('num', numeric_pipeline, numeric_cols),
    ('cat', categorical_pipeline, categorical_cols)
])

full_pipeline = Pipeline([
    ('preprocessor', preprocessor),
    ('classifier',
xgb.XGBClassifier(random_state=42))
])

full_pipeline.fit(X_train, y_train)
y_pred = full_pipeline.predict(X_test) #
preprocessing applied automatically
```

11.2 Experiment Tracking

```
import mlflow
import mlflow.sklearn

mlflow.set_experiment('my-ml-project')

with mlflow.start_run(run_name='xgboost-v2'):
    # Log parameters
    mlflow.log_params({'n_estimators': 500, 'learning_rate': 0.05, 'max_depth': 6})

    # Train
    model.fit(X_train, y_train)

    # Log metrics
    mlflow.log_metrics({
```

```

    'train_auc': roc_auc_score(y_train, model.predict_proba(X_train)[: ,1]),
    'val_auc': roc_auc_score(y_val, model.predict_proba(X_val)[: ,1]),
})

# Log model
mlflow.sklearn.log_model(model, 'model')
mlflow.log_artifact('confusion_matrix.png')

```

11.3 Model Deployment

Approach	Description & Tools
REST API (FastAPI)	Serve model predictions via HTTP endpoints. FastAPI + Uvicorn. Simple and scalable. Add authentication, rate limiting.
Batch Inference	Run predictions on large datasets periodically (e.g., nightly). Store results to database. No latency constraint.
Streaming (Kafka+Flink)	Process records as they arrive. Low latency. Complex infrastructure.
ONNX Export	Export model to Open Neural Network Exchange format. Run on any framework/hardware. Faster inference.
TorchServe / TF Serving	Production-grade model serving servers with versioning, batching, A/B testing.
Edge (TFLite, CoreML)	Deploy quantized/pruned models on mobile or IoT devices. Model must be compressed first.

```

# FastAPI model serving
from fastapi import FastAPI
from pydantic import BaseModel
import numpy as np
import joblib

app = FastAPI(title='ML Model API')
model = joblib.load('model.pkl')

class Features(BaseModel):
    feature1: float
    feature2: float
    feature3: str

@app.post('/predict')
def predict(features: Features):
    X = np.array([[features.feature1, features.feature2]])
    prob = model.predict_proba(X)[0, 1]
    pred = int(prob >= 0.5)
    return {'prediction': pred, 'probability': round(prob, 4)}

@app.get('/health')
def health(): return {'status': 'ok'}

# Run: uvicorn main:app --host 0.0.0.0 --port 8000 --workers 4

```

11.4 Handling Imbalanced Data

Technique	Description
-----------	-------------

Class weights	Pass class_weight='balanced' to sklearn models. Upweights minority class in loss function.
Oversampling (SMOTE)	Synthetic Minority Oversampling Technique. Create synthetic minority samples by interpolating between existing ones.
Undersampling	Randomly remove majority class samples. Fast but wastes data.
Threshold tuning	Lower prediction threshold from 0.5 to improve recall on minority class. Use precision-recall curve.
Anomaly detection framing	If ratio < 1:100, treat as anomaly detection (Isolation Forest, Autoencoders).

```

from imblearn.over_sampling import SMOTE
from imblearn.pipeline import Pipeline as ImbPipeline

# SMOTE pipeline
pipe = ImbPipeline([
    ('smote', SMOTE(sampling_strategy='auto', random_state=42, k_neighbors=5)),
    ('scaler', StandardScaler()),
    ('clf', LogisticRegression())
])
# NOTE: Apply SMOTE only in training folds, not on test data!
scores = cross_val_score(pipe, X_train, y_train, cv=5, scoring='f1')

# Threshold tuning
from sklearn.metrics import precision_recall_curve
prec, rec, thresholds = precision_recall_curve(y_val, y_prob)
f1_scores = 2*prec*rec / (prec+rec+1e-10)
optimal_threshold = thresholds[np.argmax(f1_scores)]
print('Optimal threshold:', optimal_threshold)

```

12. Quick Reference & Cheat Sheet

12.1 Algorithm Selection Guide

Scenario	Recommended Approach
Structured/tabular, <100K rows, interpretability needed	Logistic Regression, Ridge, Random Forest, XGBoost
Structured/tabular, any size, max accuracy	XGBoost or LightGBM + hyperparameter tuning + ensemble
High-dimensional sparse text/categorical features	Logistic Regression (L1), SVM (linear), Multinomial NB
Image classification	ResNet / EfficientNet transfer learning; ViT for large datasets
Time series forecasting	LightGBM with lag features, LSTM, Temporal Fusion Transformer
NLP: text classification	Fine-tune BERT/DistilBERT; for large-scale, few-shot GPT prompting
Anomaly detection	Isolation Forest, Autoencoder, LOF, One-Class SVM
Clustering	K-Means (convex clusters), DBSCAN (arbitrary shape + outliers), HDBSCAN
Dimensionality reduction (visualization)	UMAP (preferred) or t-SNE; PCA for linear reduction
Few labeled samples (<1000)	Transfer learning + data augmentation; semi-supervised learning
Reinforcement learning (game/control)	PPO (stable), SAC (continuous actions), DQN (discrete actions)

12.2 Common Pitfalls Checklist

- **Data leakage:** Never fit preprocessor on test data. No future features in training data.
- **Train/test split before EDA:** Explore and build features only on training set.
- **Wrong CV for time series:** Always walk-forward; never shuffle. Group-fold for grouped data.
- **Scaling before SVM/KNN/PCA:** Distance-based algorithms require scaled features.
- **Class imbalance ignored:** Use stratified split, class weights, SMOTE, PR-AUC instead of accuracy.
- **Evaluating on training data:** Always use held-out validation/test set for final metrics.
- **Multiple test set evaluations:** Each peek inflates apparent performance. Use test set only once.
- **Ignoring baseline:** Always compare against simple baselines (mean prediction, most-frequent class).
- **OHE on test introduces new categories:** Use handle_unknown='ignore' in OneHotEncoder.
- **Gradient not clipped in RNNs:** Use clip_grad_norm_ (threshold 1.0) to prevent exploding gradients.

12.3 Key Formulas Summary

Formula / Concept	Expression
Linear regression loss	$L = (1/m) \ y - Xw\ ^2$
Ridge solution	$w^* = (X^T X + \lambda I)^{-1} X^T y$
Sigmoid	$\sigma(z) = 1/(1+e^{-z})$

Softmax	$p_k = \exp(z_k) / \sum \exp(z_j)$
Cross-entropy	$L = -\sum y_i \log(\hat{y}_i)$
Backprop chain rule	$dL/dW = (dL/da)(da/dz)(dz/dW)$
Adam update	$m \leftarrow \beta_1 m + (1-\beta_1)g; v \leftarrow \beta_2 v + (1-\beta_2)g^2; w \leftarrow w - \eta \cdot \hat{m} / \sqrt{\hat{v} + \epsilon}$
PCA eigendecomposition	$C = Q\Lambda Q^T; Z = XQ$
KL divergence	$KL(P Q) = \sum P(x)\log[P(x)/Q(x)]$
Transformer attention	$Attn(Q,K,V) = \text{softmax}(QK^T/\sqrt{d})V$
Bellman equation	$V^*(s) = \max_a [r(s,a) + \gamma \sum P(s' s,a)V^*(s')]$
ELBO (VAE)	$E[\log P(x z)] - KL(Q(z x) P(z))$
Bayes theorem	$P(\theta D) \propto P(D \theta) \cdot P(\theta)$
F1 Score	$2 \cdot P \cdot R / (P+R)$
R ²	$1 - SS_{res}/SS_{tot}$

End of Notes — Good luck! ☐